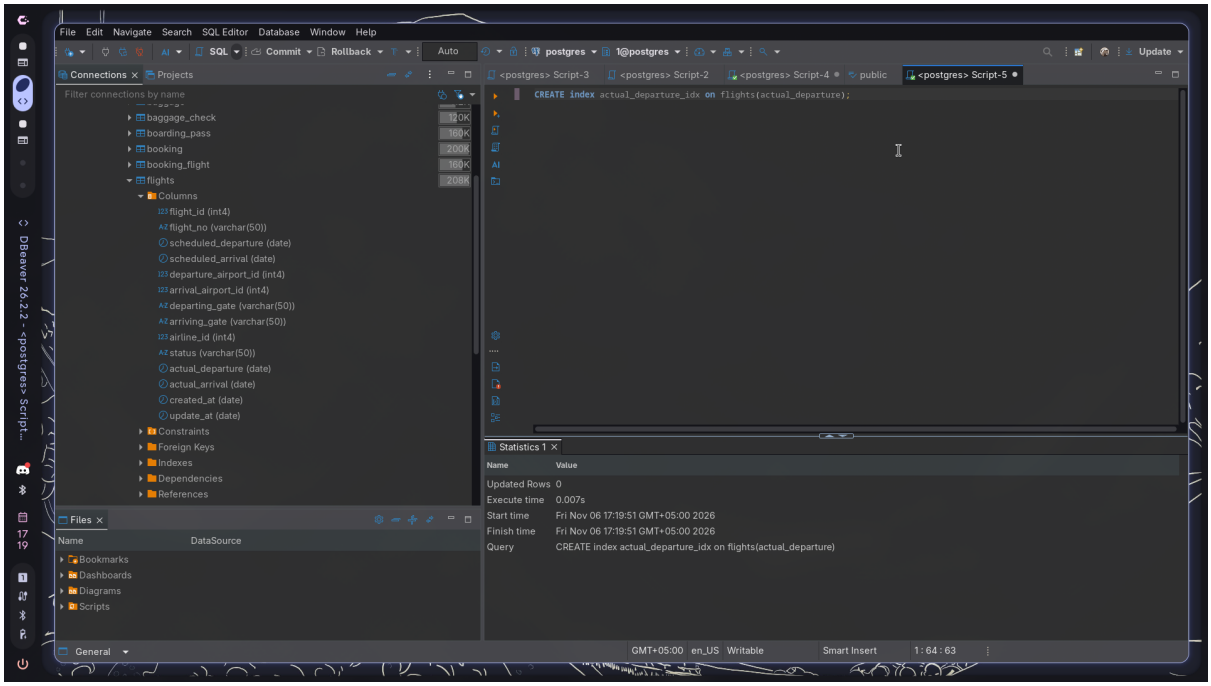
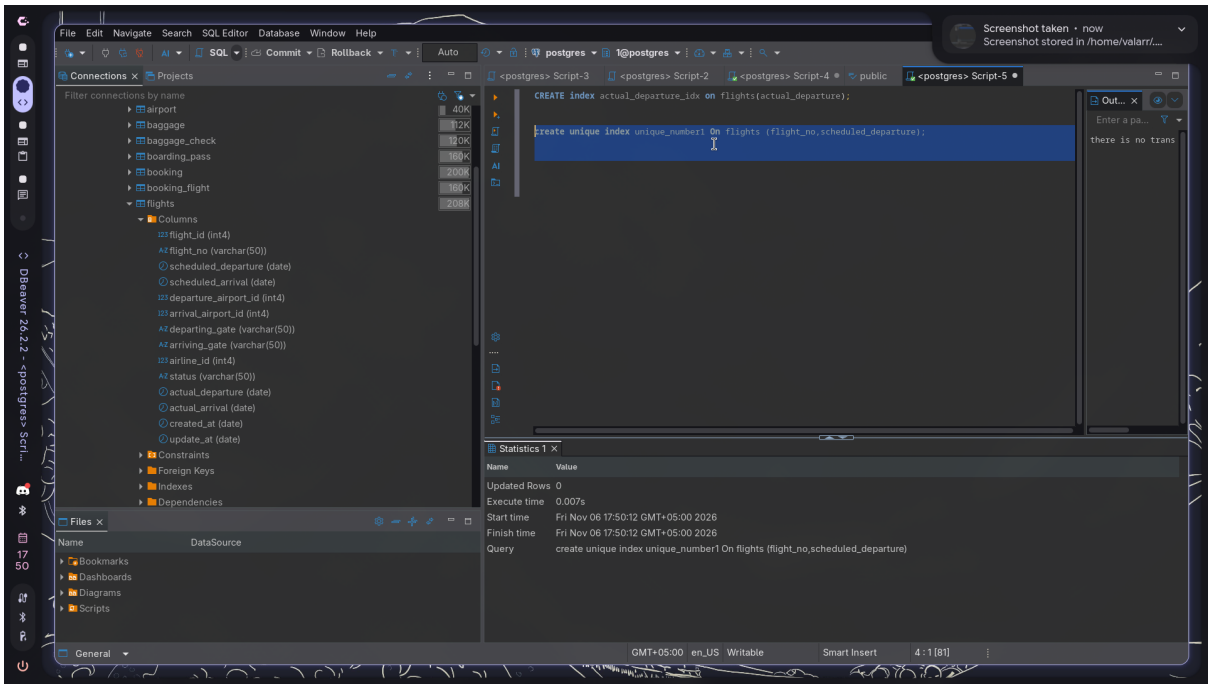
lab7

Alnur bekmurza

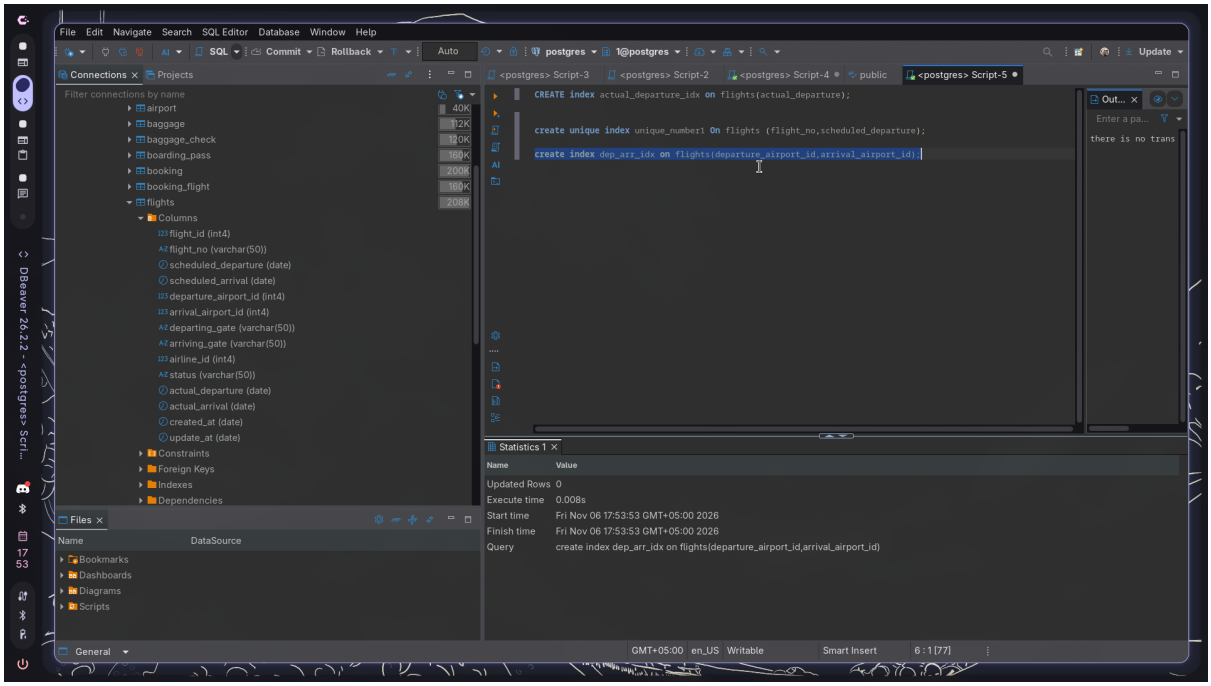
1)



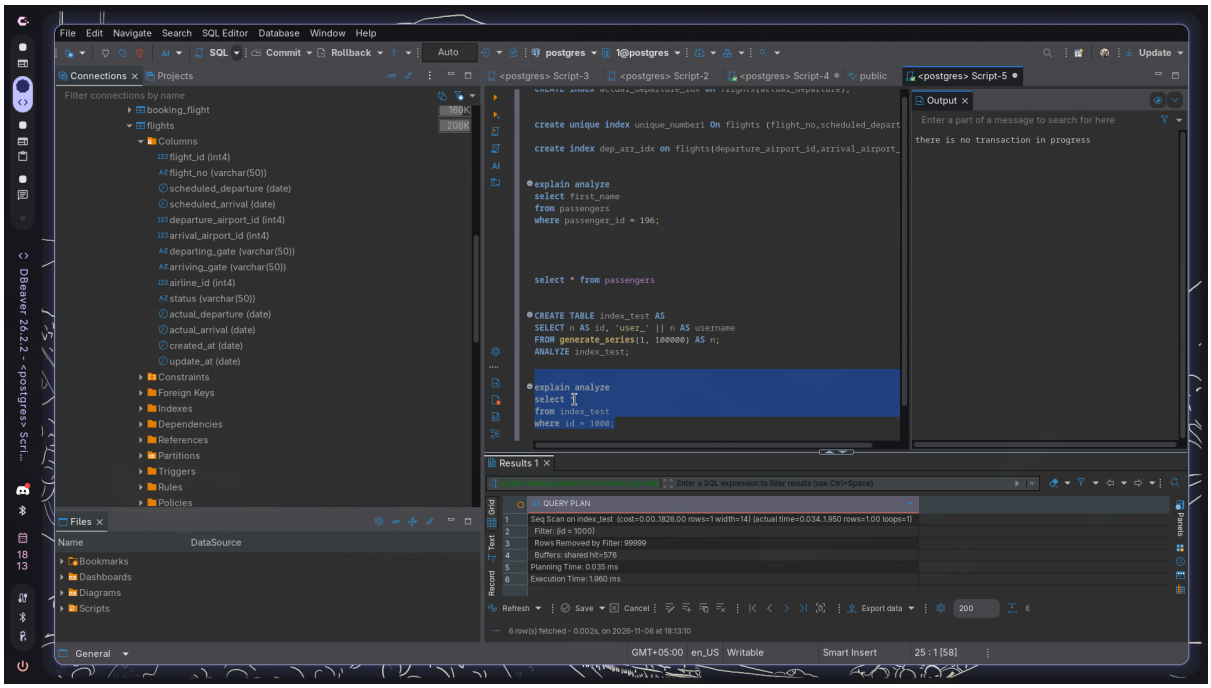
2)

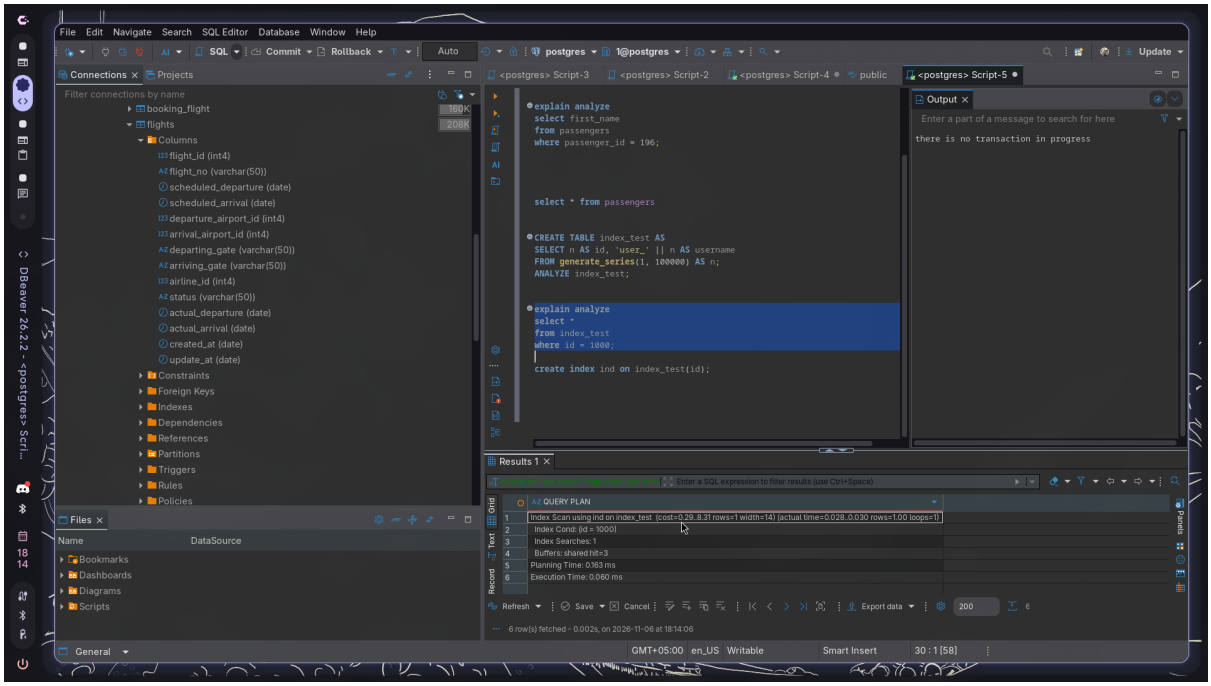


3)

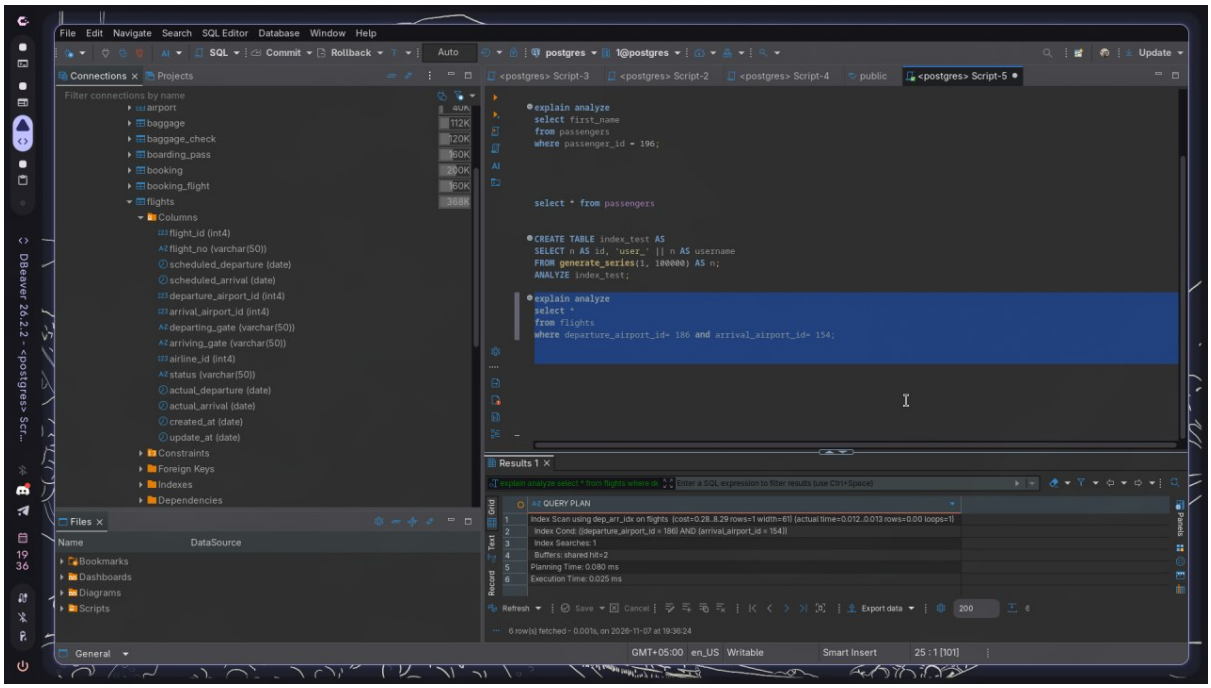

File Edit Navigate Search SQL Editor Database Window Help
CREATE index actual_departure_idx on flights(actual_departure);
create unique index unique_number1 On flights (flight_no,scheduled_departure);
create index dep_arr_idx on flights(departure_airport_id,arrival_airport_id);
there is no trans
Statistics 1
Name Value
Updated Rows 0
Execute time 0.008s
Start time Fri Nov 06 17:53:53 GMT+05:00 2026
Finish time Fri Nov 06 17:53:53 GMT+05:00 2026
Query create index dep_arr_idx on flights(departure_airport_id,arrival_airport_id)

4)

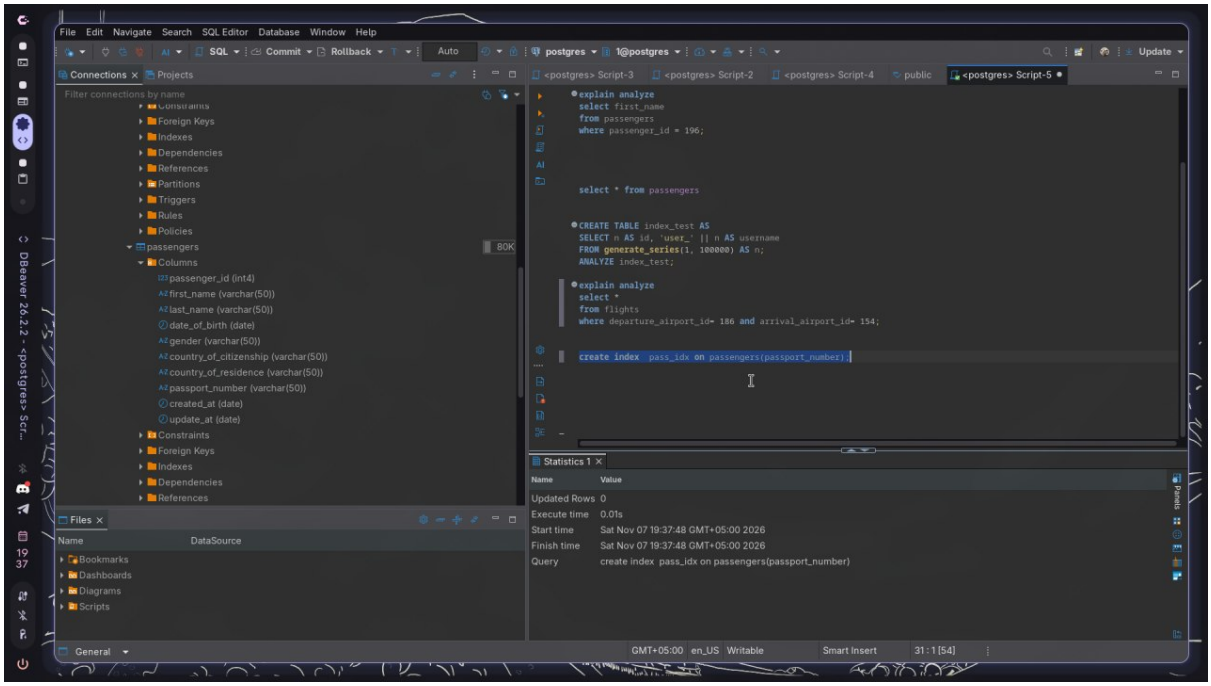

File Edit Navigate Search SQL Editor Database Window Help
create unique index unique_number1 On flights (flight_no,scheduled_depart
create index dep_arr_idx on flights(departure_airport_id,arrival_airport_
explain analyze
select first_name
from passengers
where passenger_id = 196;
select * from passengers
CREATE TABLE index_test AS
SELECT n AS id, 'user_' || n AS username
FROM generate_series(1, 100000) AS n;
ANALYZE index_test;
explain analyze
select
from index_test
where id = 1000;
there is no transaction in progress
Results 1
QUERY PLAN
Seq Scan on index_test (cost=0.00..1826.00 rows=1 width=14) (actual time=0.034..1.950 rows=1.00 loops=1)
Filter: (id = 1000)
Rows Removed by Filter: 99999
Buffers: shared hit=576
Planning Time: 0.035 ms
Execution Time: 1.960 ms
6 row(s) fetched - 0.002s, on 2026-11-06 at 18:13:10

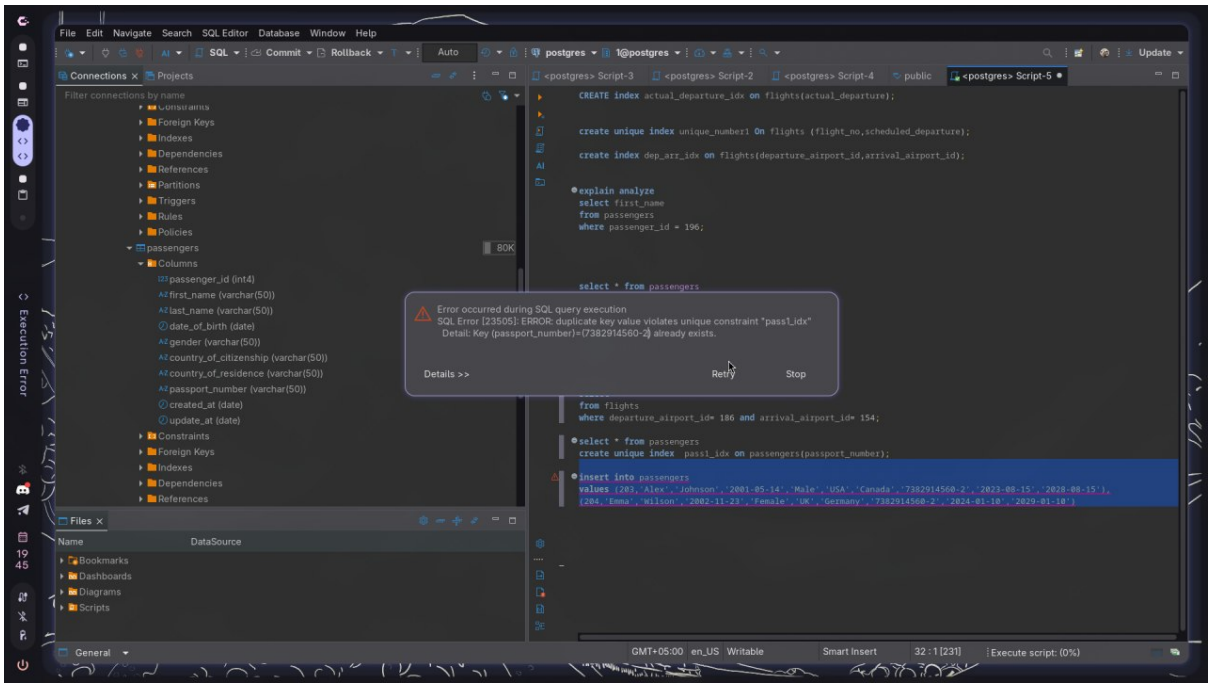

File Edit Navigate Search SQL Editor Database Window Help
explain analyze
select first_name
from passengers
where passenger_id = 196;
select * from passengers
CREATE TABLE index_test AS
SELECT n AS id, 'user_' || n AS username
FROM generate_series(1, 100000) AS n;
ANALYZE index_test;
explain analyze
select *
from index_test
where id = 1000;
create index ind on index_test(id);
there is no transaction in progress
Index Scan using ind on index_test (cost=0.29..8.31 rows=1 width=14) (actual time=0.028..0.030 rows=1.00 loops=1)
Index Cond: (id = 1000)
Index Searches: 1
Buffers: shared hit=3
Planning Time: 0.163 ms
Execution Time: 0.060 ms

5)


File Edit Navigate Search SQL Editor Database Window Help
explain analyze
select first_name
from passengers
where passenger_id = 196;
select * from passengers
CREATE TABLE index_test AS
SELECT n AS id, 'user_' || n AS username
FROM generate_series(1, 100000) AS n;
ANALYZE index_test;
explain analyze
select *
from flights
where departure_airport_id= 186 and arrival_airport_id= 154;
Index Scan using dep_arr_idx on flights (cost=0.28..8.29 rows=1 width=61) (actual time=0.012..0.013 rows=0.00 loops=1)
Index Cond: ((departure_airport_id = 186) AND (arrival_airport_id = 154))
Index Searches: 1
Buffers: shared hit=2
Planning Time: 0.080 ms
Execution Time: 0.025 ms

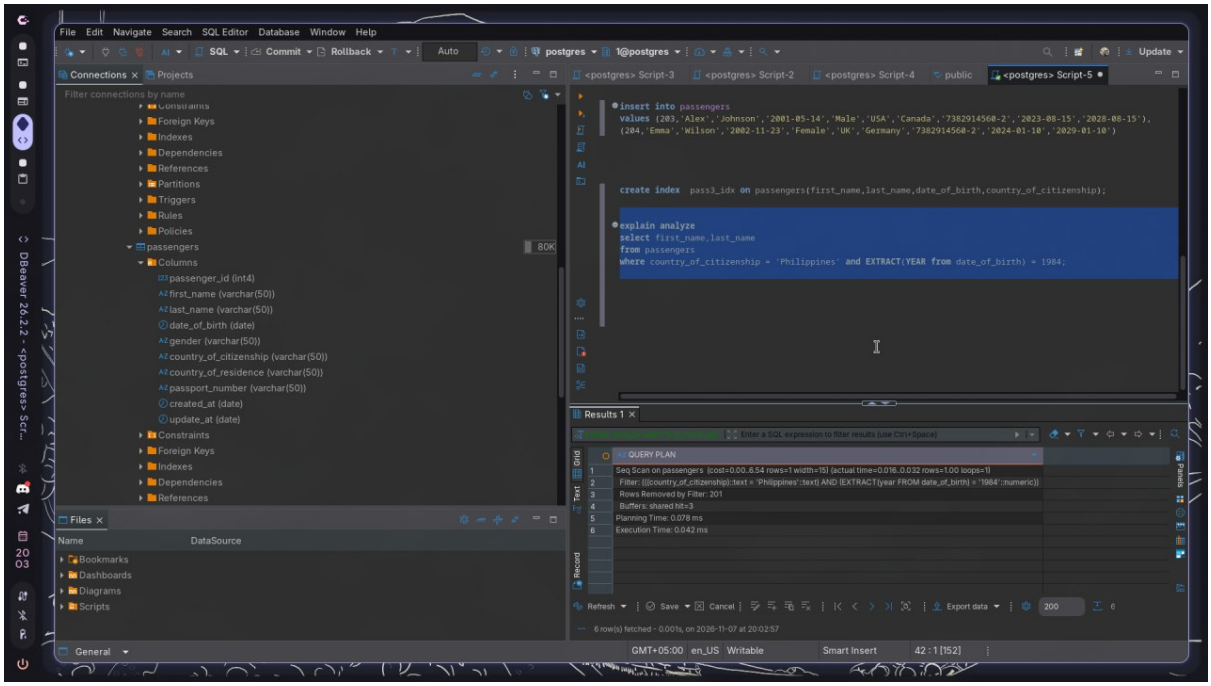
6)



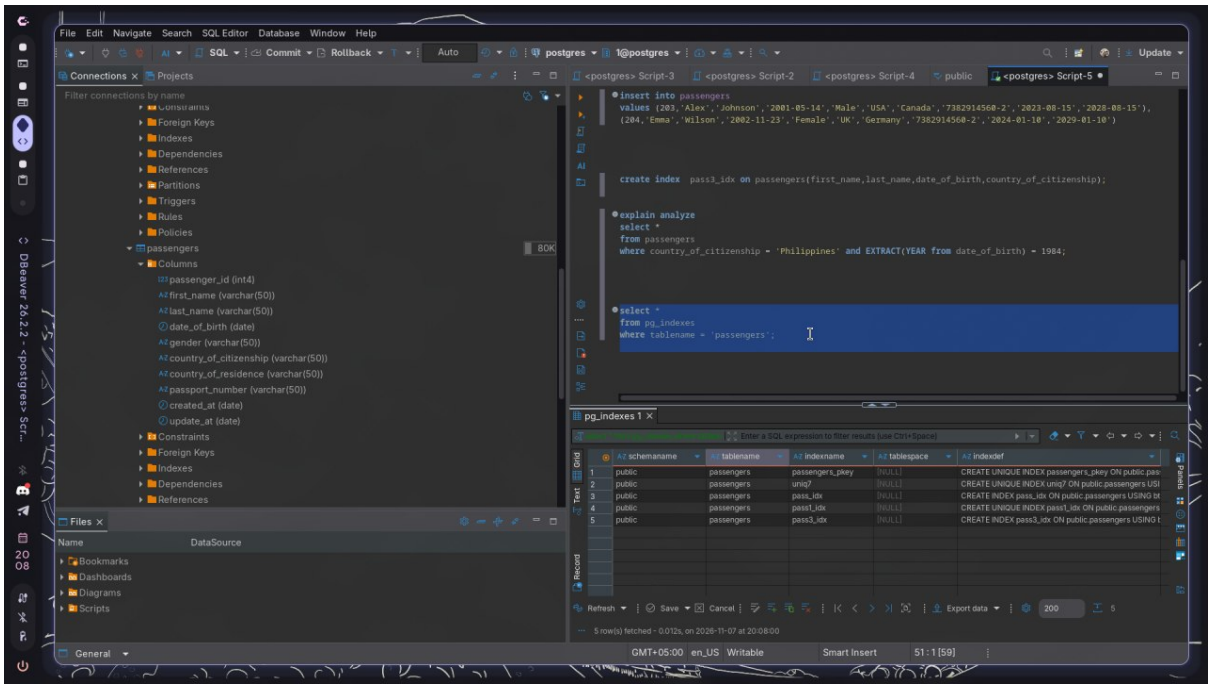


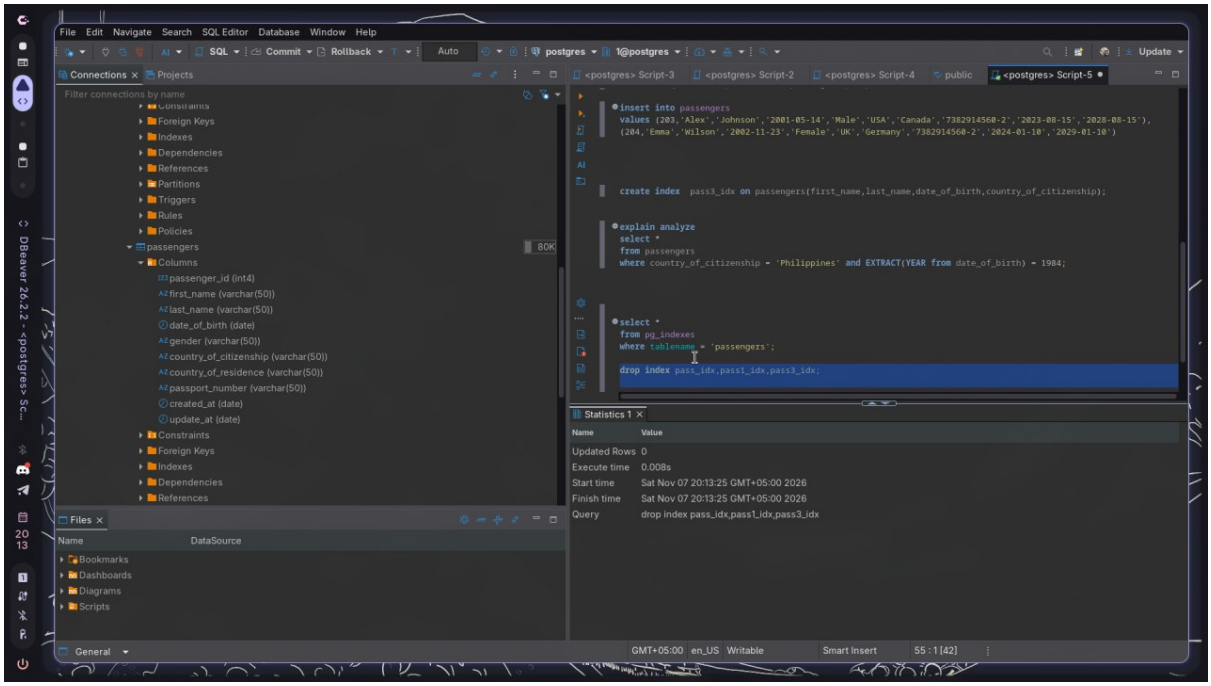
The unique index checks that each non-null passport number is unique. When trying to insert a passenger with a new passport number, the insert succeeds. If I try to insert a passport number that already exists, psql rejects it to prevent duplicates.

7)

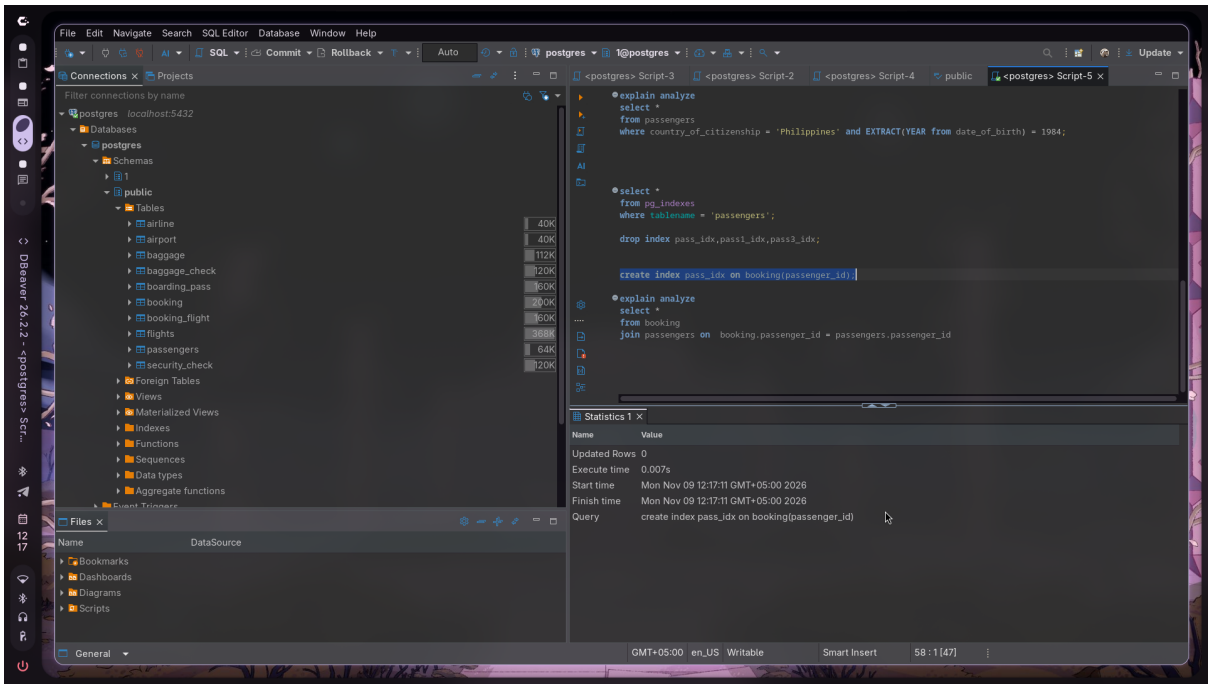

insert into passengers
values (203,'Alex','Johnson','2001-05-14','Male','USA','Canada','7382914560-2','2023-08-15','2028-08-15'),
(204,'Emma','Wilson','2002-11-23','Female','UK','Germany','7382914560-2','2024-01-10','2029-01-10')
create index pass3_idx on passengers(first_name,last_name,date_of_birth,country_of_citizenship);
explain analyze
select first_name,last_name
from passengers
where country_of_citizenship = 'Philippines' and EXTRACT(YEAR from date_of_birth) = 1984;
QUERY PLAN
Seq Scan on passengers (cost=0.00..8.54 rows=1 width=15) (actual time=0.016..0.032 rows=1 loops=1)
Filter: (((country_of_citizenship)::text = 'Philippines'::text) AND (EXTRACT(year FROM date_of_birth) = '1984'::numeric))
Rows Removed by Filter: 201
Buffers: shared hit=3
Planning Time: 0.078 ms
Execution Time: 0.042 ms

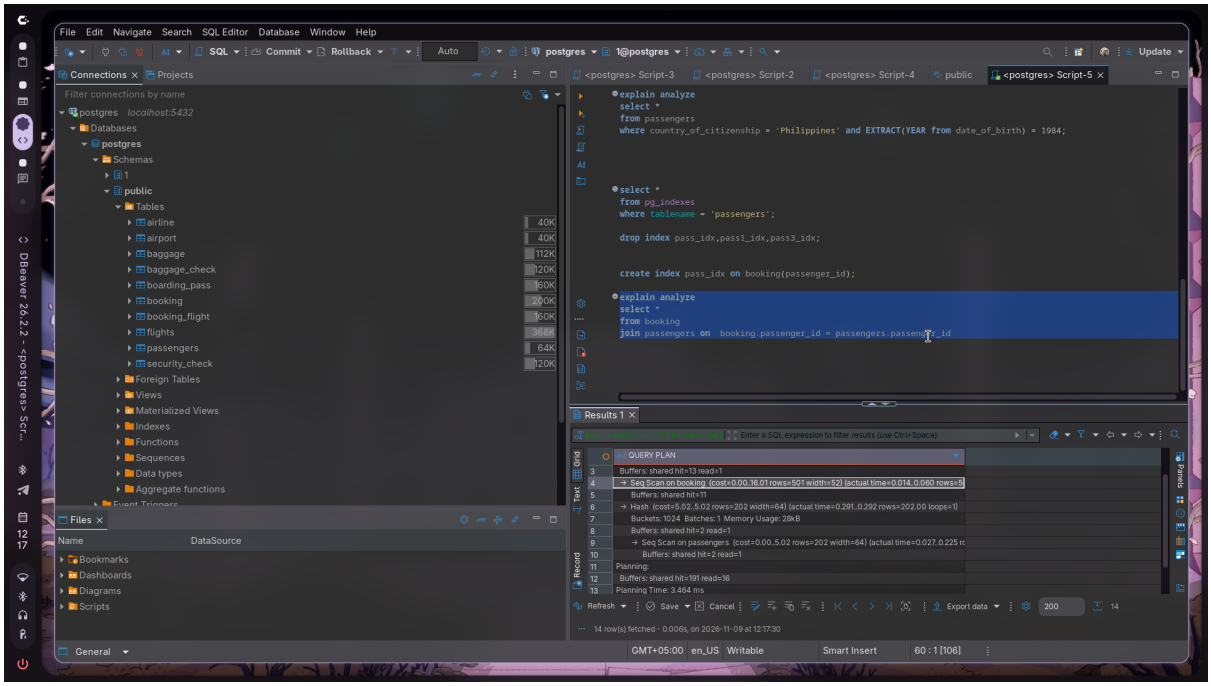
8)


insert into passengers
values (203,'Alex','Johnson','2001-05-14','Male','USA','Canada','7382914560-2','2023-08-15','2028-08-15'),
(204,'Emma','Wilson','2002-11-23','Female','UK','Germany','7382914560-2','2024-01-10','2029-01-10')
create index pass3_idx on passengers(first_name,last_name,date_of_birth,country_of_citizenship);
explain analyze
select *
from passengers
where country_of_citizenship = 'Philippines' and EXTRACT(YEAR from date_of_birth) = 1984;
select *
from pg_indexes
where tablename = 'passengers';
schemaname
tablename
indexname
tablespace
indexdef
public passengers passengers_pkey [NULL] CREATE UNIQUE INDEX passengers_pkey ON public.pass
public passengers uniq7 [NULL] CREATE UNIQUE INDEX uniq7 ON public.passengers USI
public passengers pass_idx [NULL] CREATE INDEX pass_idx ON public.passengers USING bt
public passengers pass1_idx [NULL] CREATE UNIQUE INDEX pass1_idx ON public.passengers
public passengers pass3_idx [NULL] CREATE INDEX pass3_idx ON public.passengers USING b

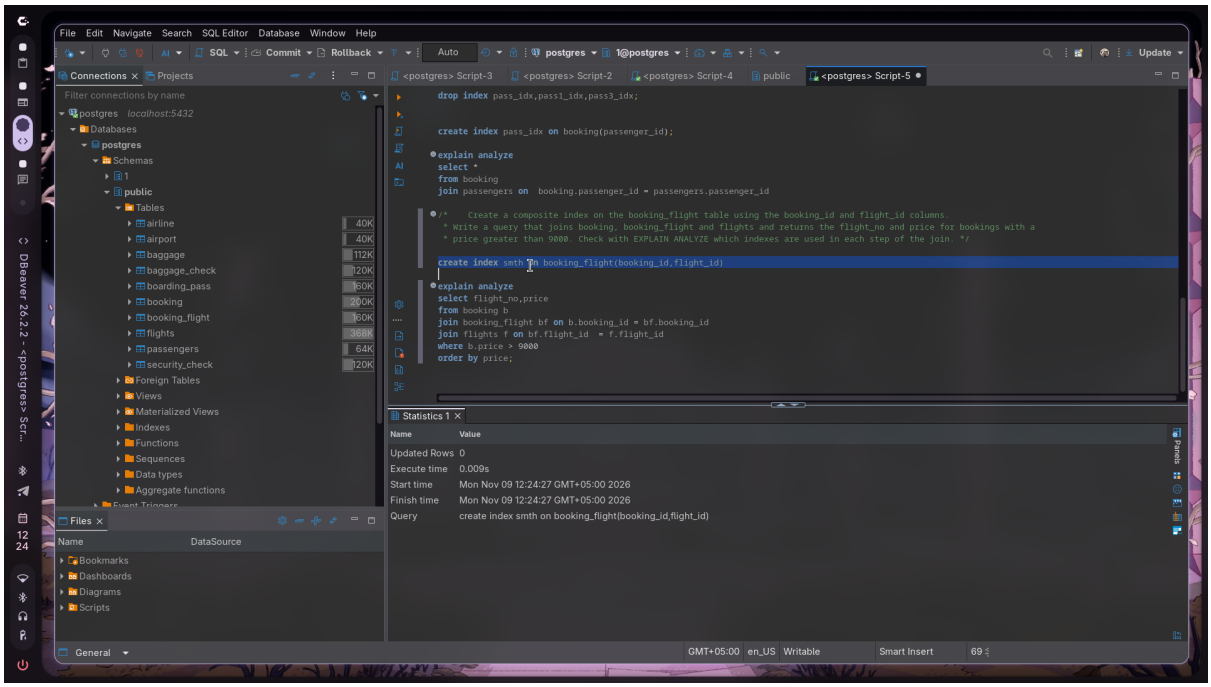

insert into passengers
values (203,'Alex','Johnson','2001-05-14','Male','USA','Canada','7382914560-2','2023-08-15','2028-08-15'),
(204,'Emma','Wilson','2002-11-23','Female','UK','Germany','7382914560-2','2024-01-10','2029-01-10')
create index pass3_idx on passengers(first_name,last_name,date_of_birth,country_of_citizenship);
explain analyze
select *
from passengers
where country_of_citizenship = 'Philippines' and EXTRACT(YEAR from date_of_birth) = 1984;
select *
from pg_indexes
where tablename = 'passengers';
drop index pass_idx,pass1_idx,pass3_idx;
Statistics 1
Name Value
Updated Rows 0
Execute time 0.008s
Start time Sat Nov 07 20:13:25 GMT+05:00 2026
Finish time Sat Nov 07 20:13:25 GMT+05:00 2026
Query drop index pass_idx,pass1_idx,pass3_idx

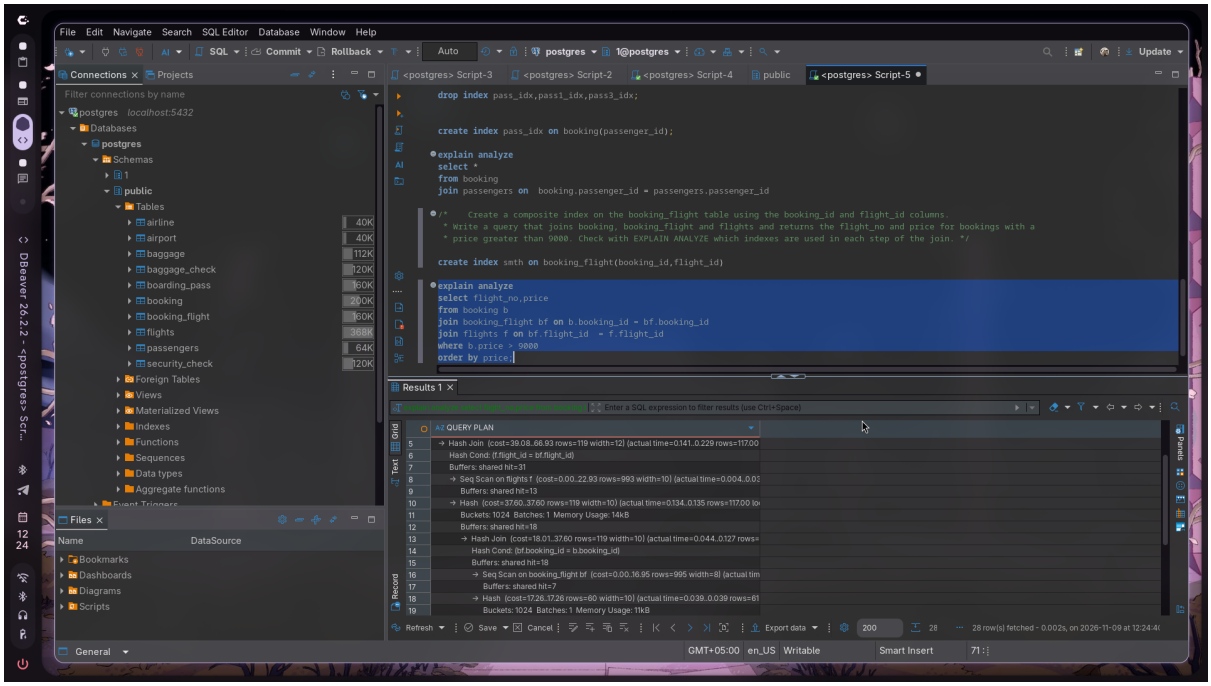
9)


explain analyze
select *
from passengers
where country_of_citizenship = 'Philippines' and EXTRACT(YEAR from date_of_birth) = 1984;
select *
from pg_indexes
where tablename = 'passengers';
drop index pass_idx,pass1_idx,pass3_idx;
create index pass_idx on booking(passenger_id);
explain analyze
select *
from booking
join passengers on booking.passenger_id = passengers.passenger_id
Statistics 1
Name Value
Updated Rows 0
Execute time 0.007s
Start time Mon Nov 09 12:17:11 GMT+05:00 2026
Finish time Mon Nov 09 12:17:11 GMT+05:00 2026
Query create index pass_idx on booking(passenger_id)

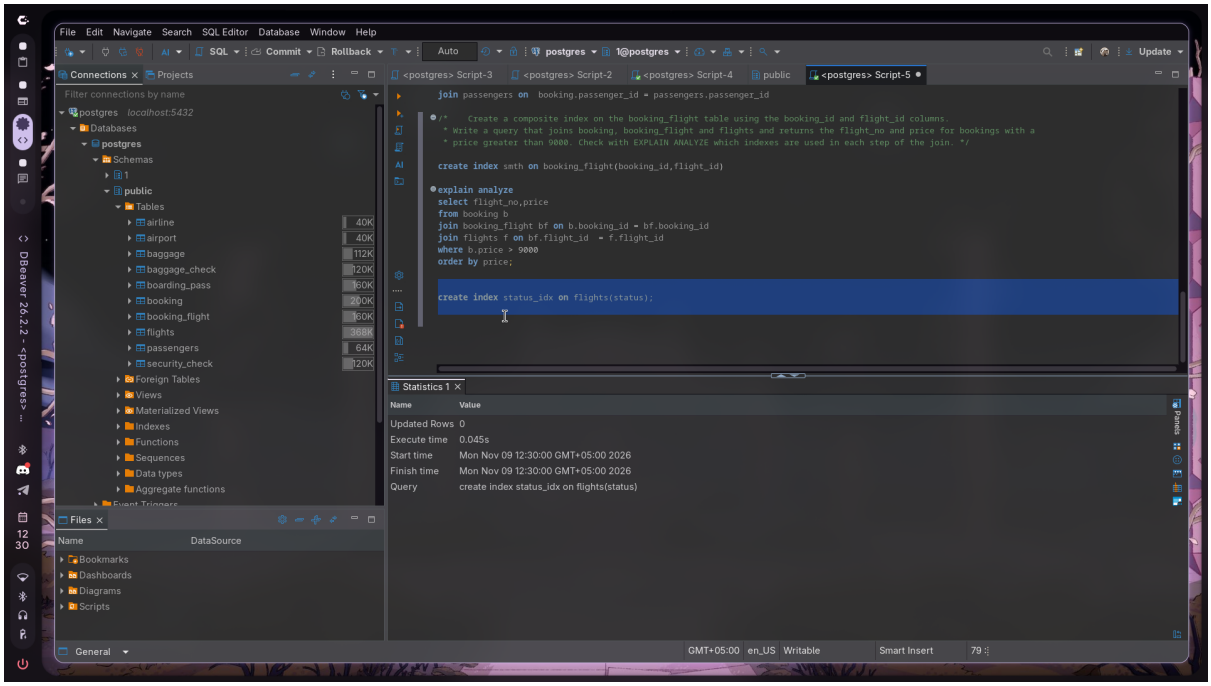

explain analyze
select *
from passengers
where country_of_citizenship = 'Philippines' and EXTRACT(YEAR from date_of_birth) = 1984;
select *
from pg_indexes
where tablename = 'passengers';
drop index pass_idx,pass1_idx,pass3_idx;
create index pass_idx on booking(passenger_id);
explain analyze
select *
from booking
join passengers on booking.passenger_id = passengers.passenger_id
Results 1
QUERY PLAN
Buffers: shared hit=13 read=1
→ Seq Scan on booking (cost=0.00..16.01 rows=501 width=52) (actual time=0.014..0.060 rows=5
Buffers: shared hit=11
→ Hash (cost=5.02..5.02 rows=202 width=64) (actual time=0.291..0.292 rows=202.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 28kB
Buffers: shared hit=2 read=1
→ Seq Scan on passengers (cost=0.00..5.02 rows=202 width=64) (actual time=0.027..0.225 r
Buffers: shared hit=2 read=1
Planning:
Buffers: shared hit=191 read=16
Planning Time: 3.464 ms
14 row(s) fetched - 0.006s, on 2026-11-09 at 12:17:30

10)

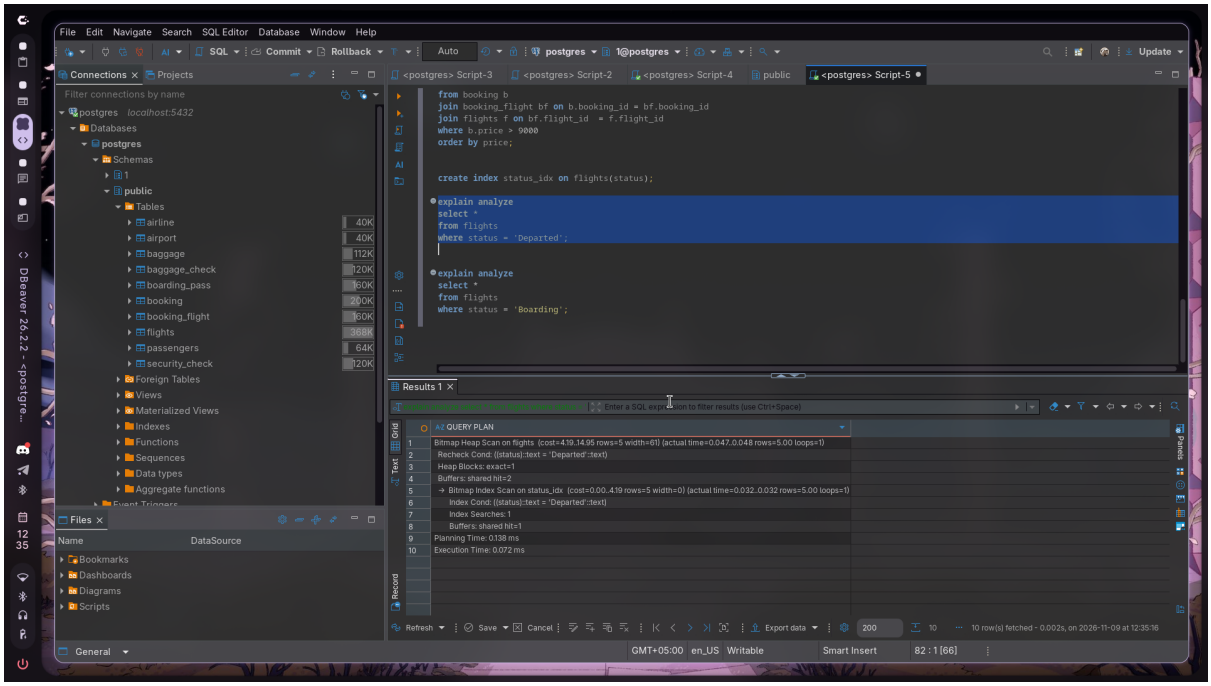

drop index pass_idx,pass1_idx,pass3_idx;
create index pass_idx on booking(passenger_id);
explain analyze
select *
from booking
join passengers on booking.passenger_id = passengers.passenger_id
/* Create a composite index on the booking_flight table using the booking_id and flight_id columns.
* Write a query that joins booking, booking_flight and flights and returns the flight_no and price for bookings with a
* price greater than 9000. Check with EXPLAIN ANALYZE which indexes are used in each step of the join. */
create index smth on booking_flight(booking_id,flight_id)
explain analyze
select flight_no,price
from booking b
join booking_flight bf on b.booking_id = bf.booking_id
join flights f on bf.flight_id = f.flight_id
where b.price > 9000
order by price;
Statistics 1
Name Value
Updated Rows 0
Execute time 0.009s
Start time Mon Nov 09 12:24:27 GMT+05:00 2026
Finish time Mon Nov 09 12:24:27 GMT+05:00 2026
Query create index smth on booking_flight(booking_id,flight_id)



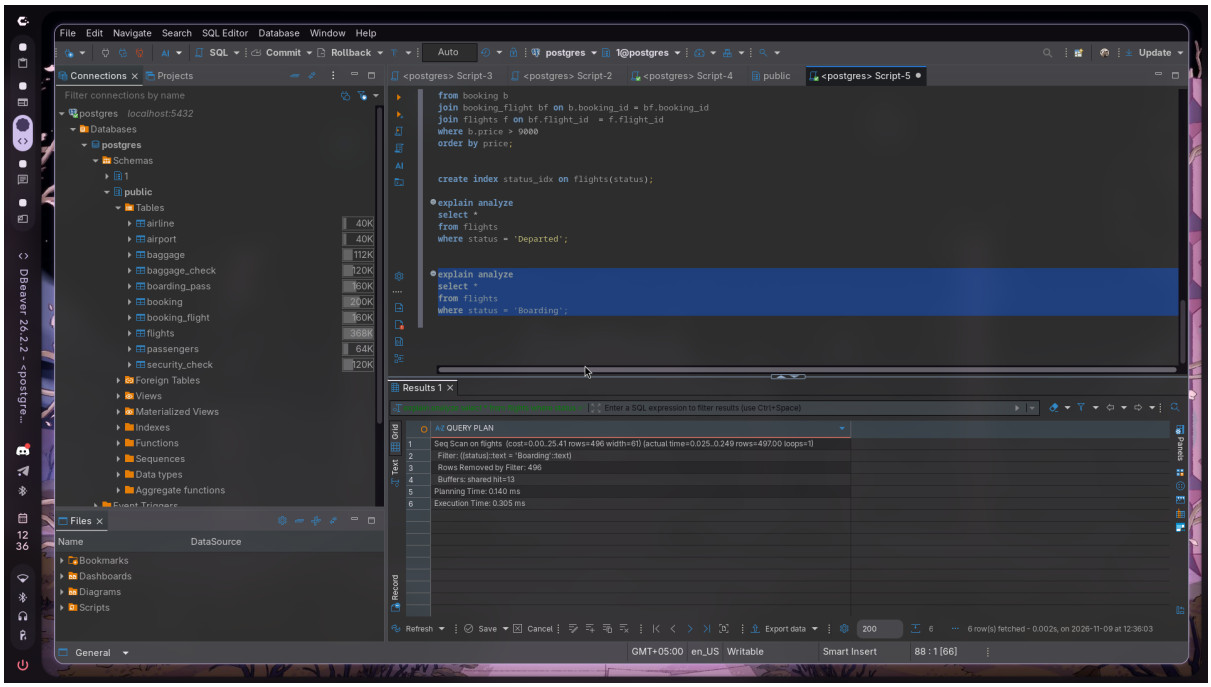
11)



Searching the status='Departed'(uses index bitmap heap search)

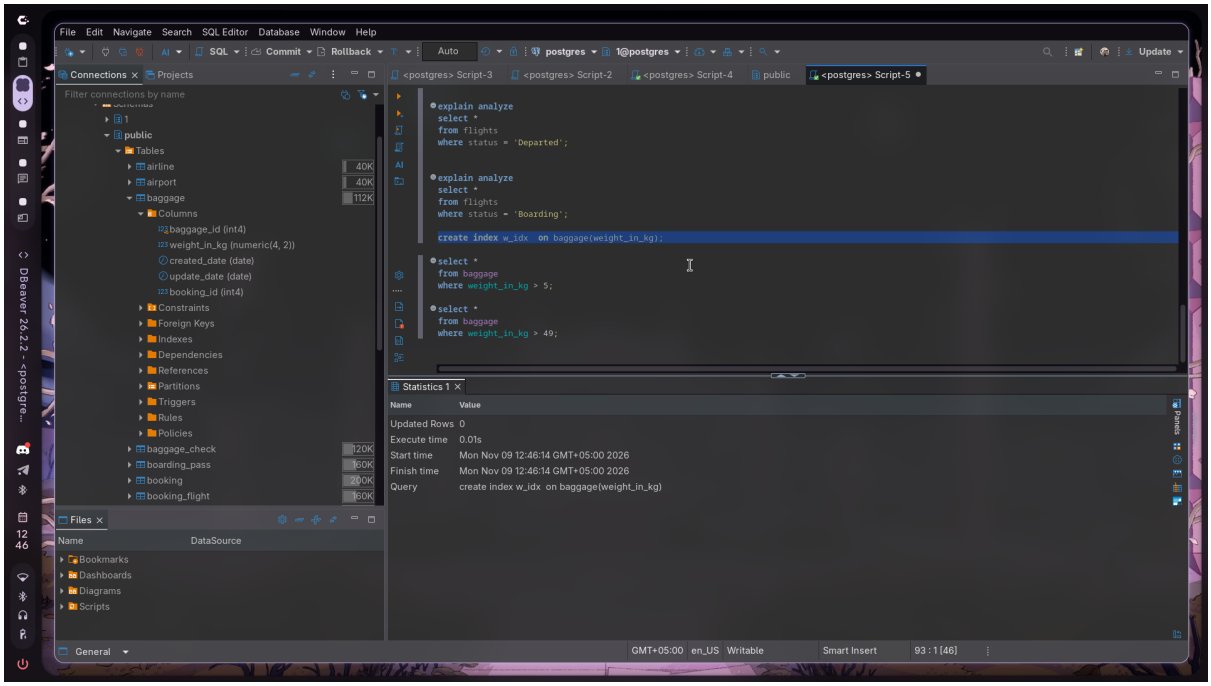


Searching status=’Boarding’(uses sequential scan) as we can notice exectuion time takes 0.305 ms while the other one used bitmap and got 0.072ms

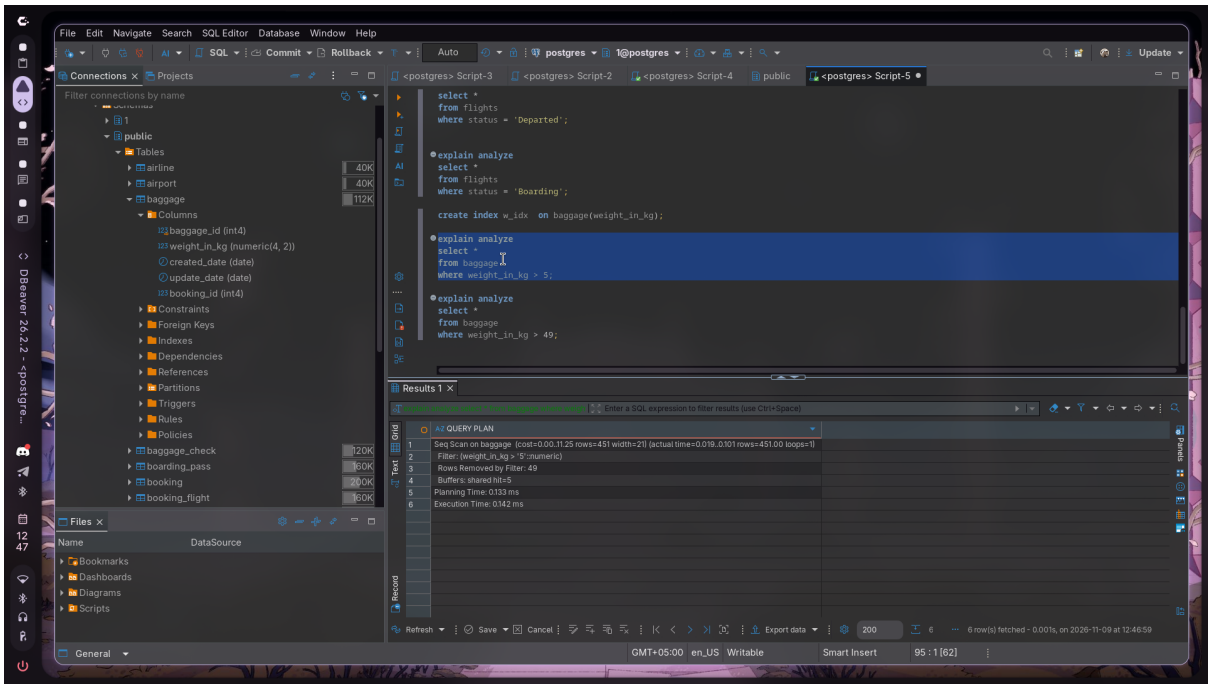


It used the seq scan if the most rows contain the boarding like in our case due to low selectivity

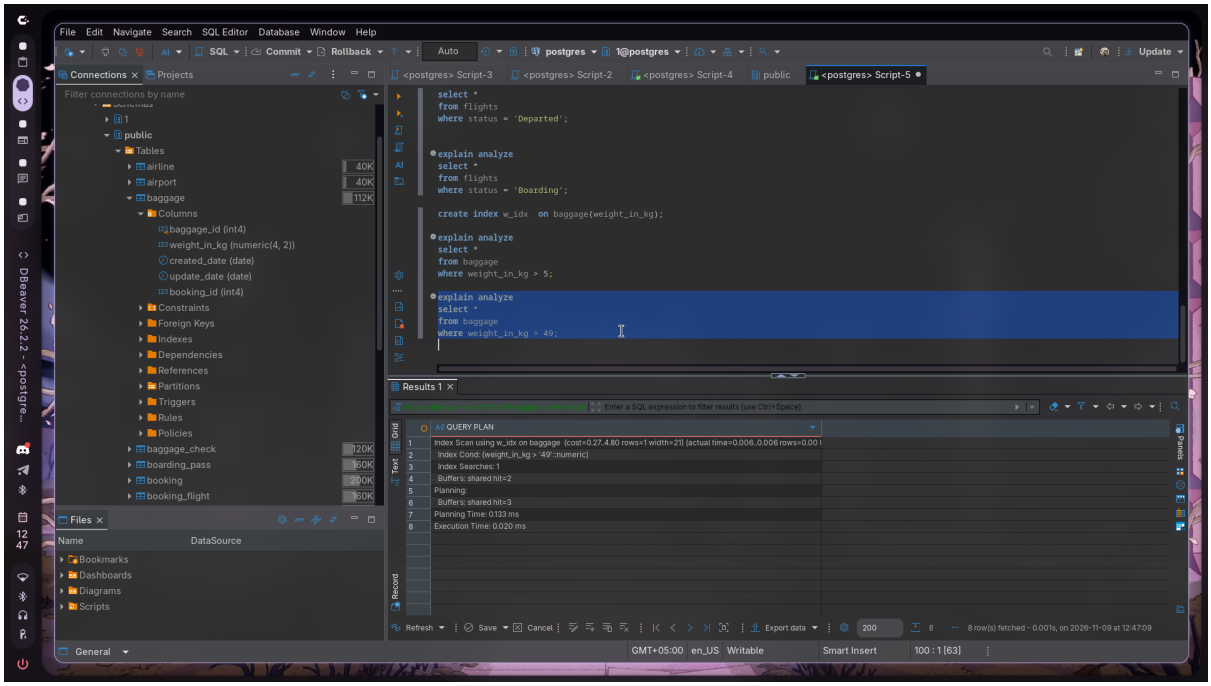
12)



Checking weight > 5

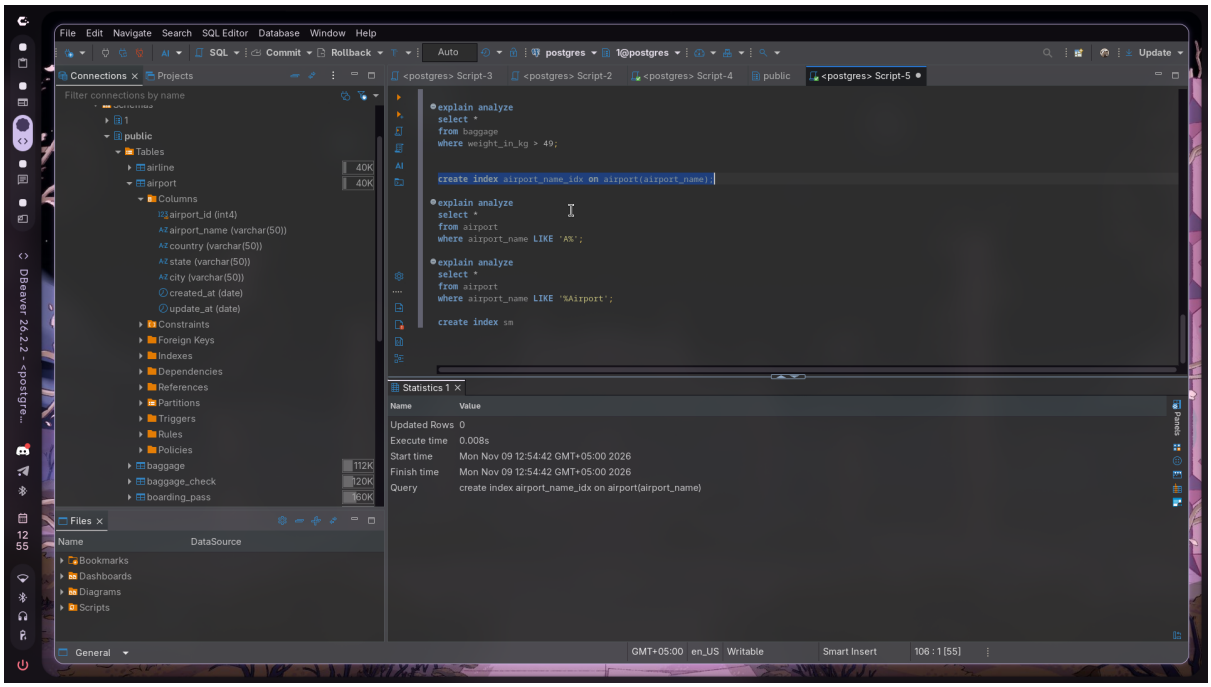


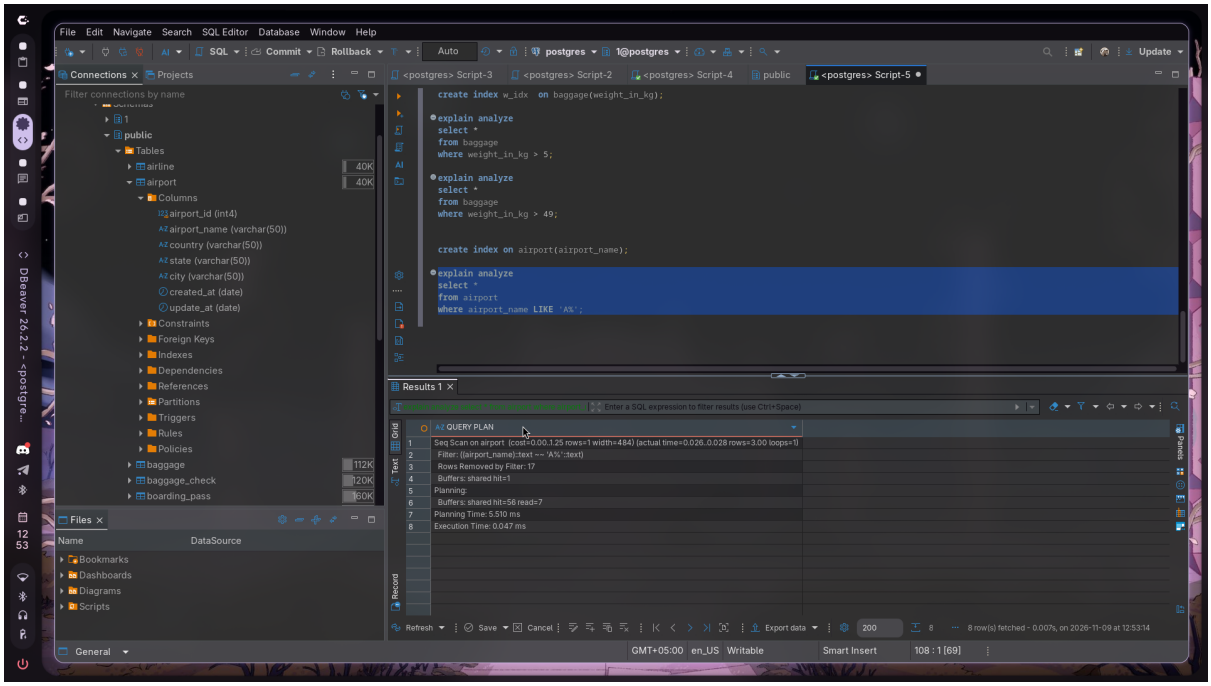
Checking weight > 49

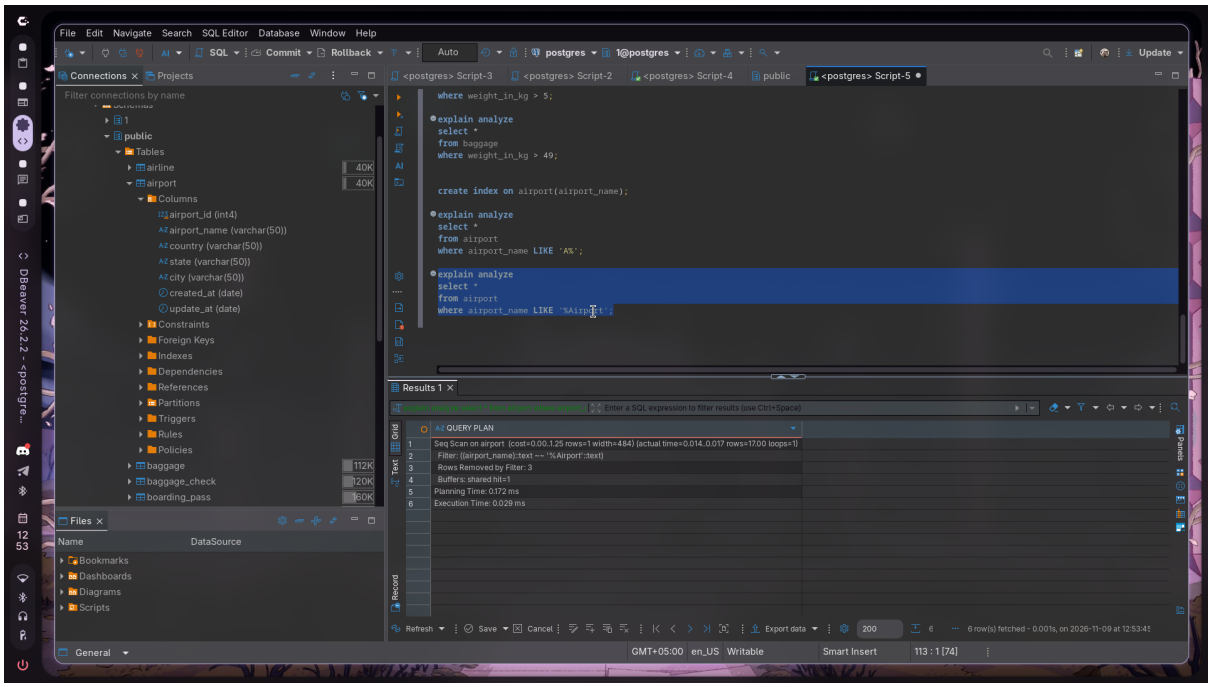


The reason postgresql chose the seq scan on weight > 5 because it rows are most of that table having more 5 kg so it chose seq to check every row, and it used index search on more than 49 due to small amount of rows that more than 49kg so its efficient to use index with less amount of rows that it can check and find.

13)

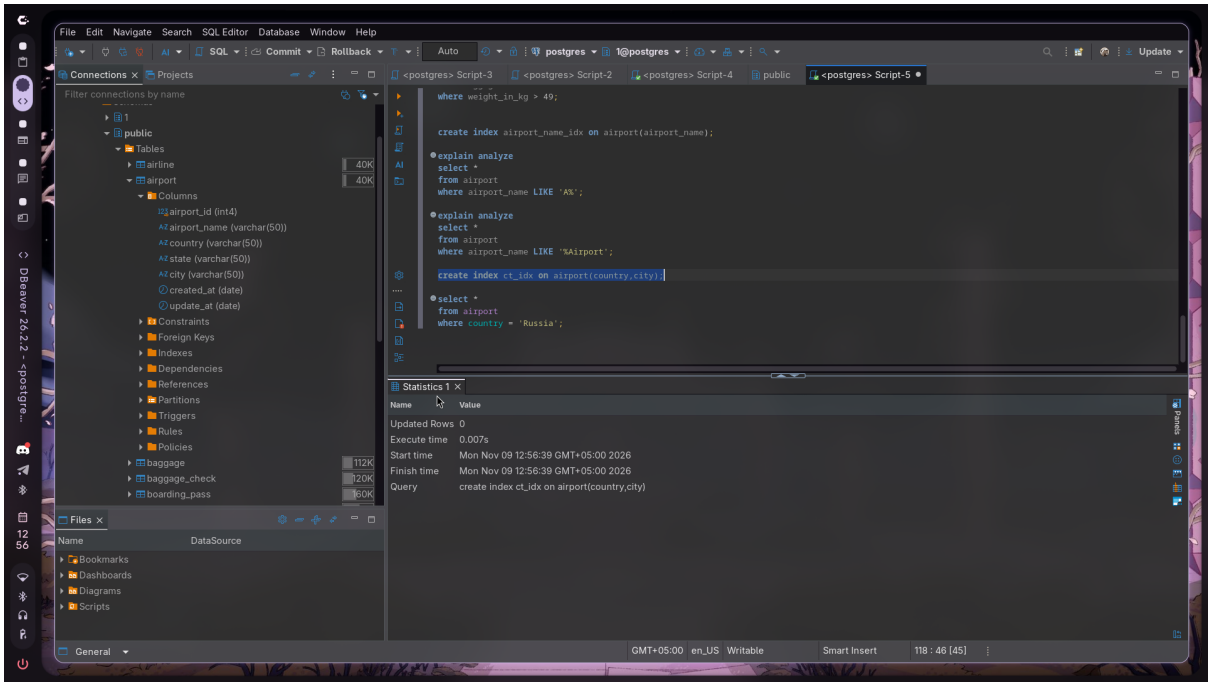


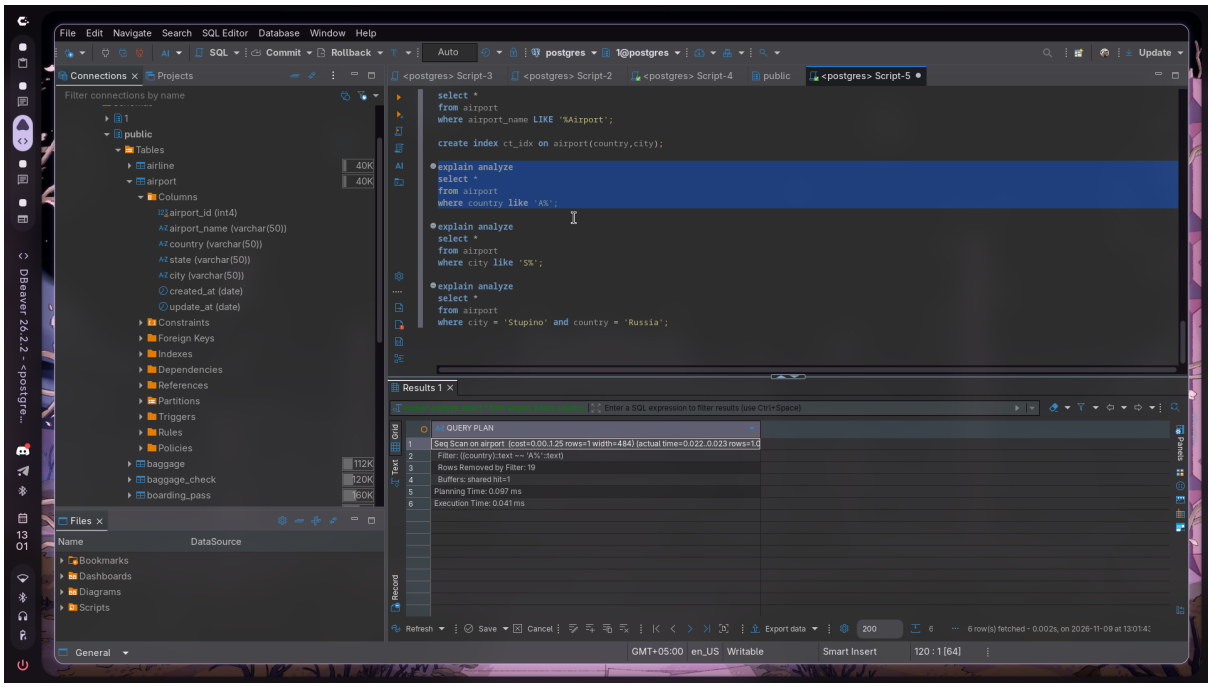


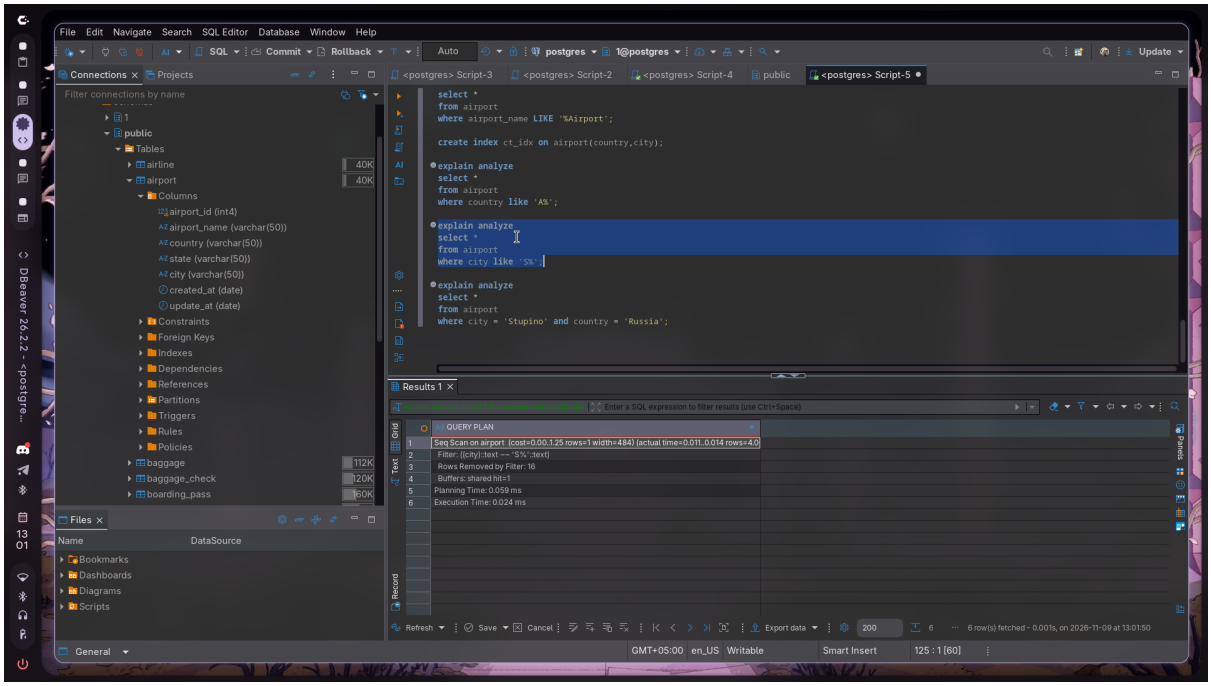


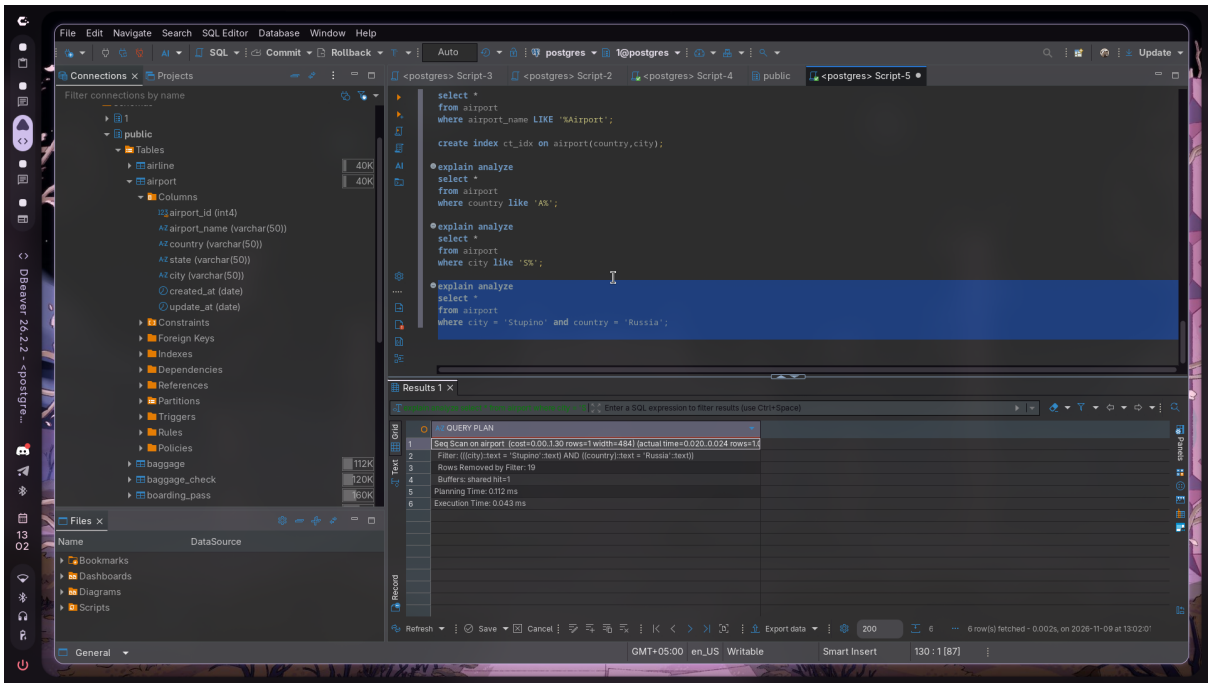
Both queries used a sequential scan.

14)


File Edit Navigate Search SQL Editor Database Window Help
Connections
Projects
<postgres> Script-3
<postgres> Script-2
<postgres> Script-4
public
<postgres> Script-5
where weight_in_kg > 49;
create index airport_name_idx on airport(airport_name);
explain analyze
select *
from airport
where airport_name LIKE 'A%';
explain analyze
select *
from airport
where airport_name LIKE '%Airport';
create index ct_idx on airport(country,city);
select *
from airport
where country = 'Russia';
Statistics 1
Name
Value
Updated Rows 0
Execute time 0.007s
Start time Mon Nov 09 12:56:39 GMT+05:00 2026
Finish time Mon Nov 09 12:56:39 GMT+05:00 2026
Query create index ct_idx on airport(country,city)
Files
Name
DataSource
Bookmarks
Dashboards
Diagrams
Scripts
General
GMT+05:00 en_US Writable Smart Insert 118 : 46 [45]

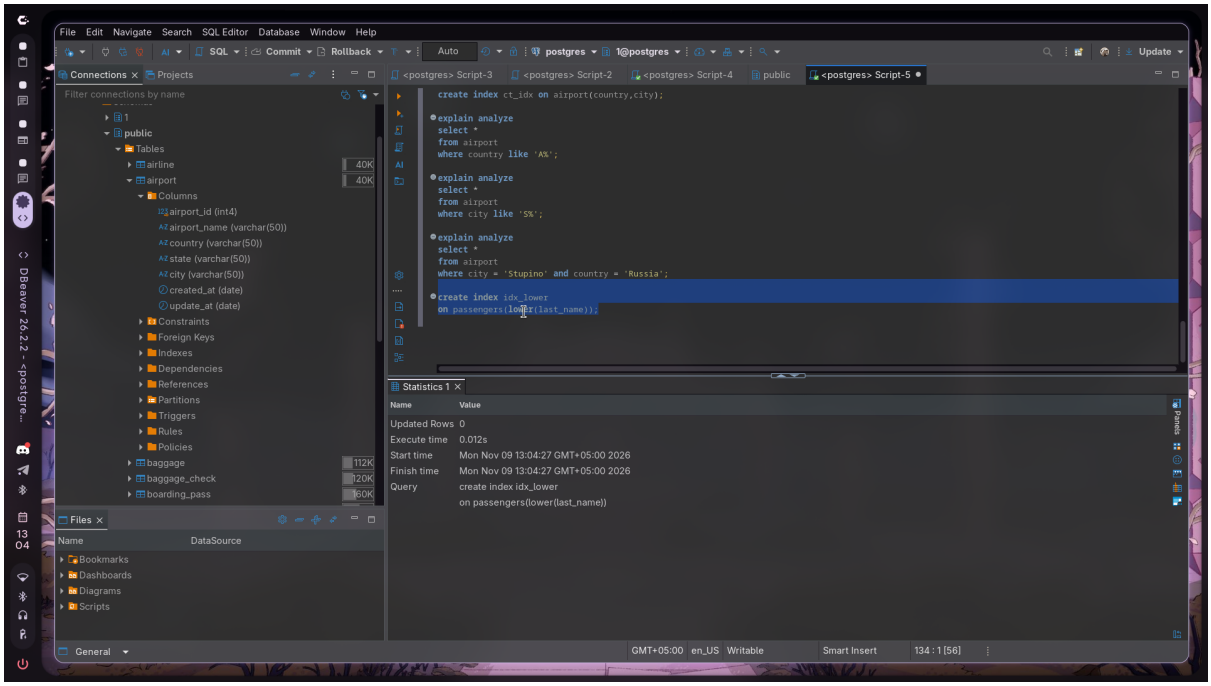

File Edit Navigate Search SQL Editor Database Window Help
Connections
Projects
<postgres> Script-3
<postgres> Script-2
<postgres> Script-4
public
<postgres> Script-5
select *
from airport
where airport_name LIKE '%Airport';
create index ct_idx on airport(country,city);
explain analyze
select *
from airport
where country like 'A%';
explain analyze
select *
from airport
where city like 'S%';
explain analyze
select *
from airport
where city = 'Stupino' and country = 'Russia';
Results 1
QUERY PLAN
Seq Scan on airport (cost=0.00..1.25 rows=1 width=484) (actual time=0.022..0.023 rows=1
Filter: ((country)::text ~~ 'A%'::text)
Rows Removed by Filter: 19
Buffers: shared hit=1
Planning Time: 0.097 ms
Execution Time: 0.041 ms
Files
Name
DataSource
Bookmarks
Dashboards
Diagrams
Scripts
General
GMT+05:00 en_US Writable Smart Insert 120 : 1 [64]

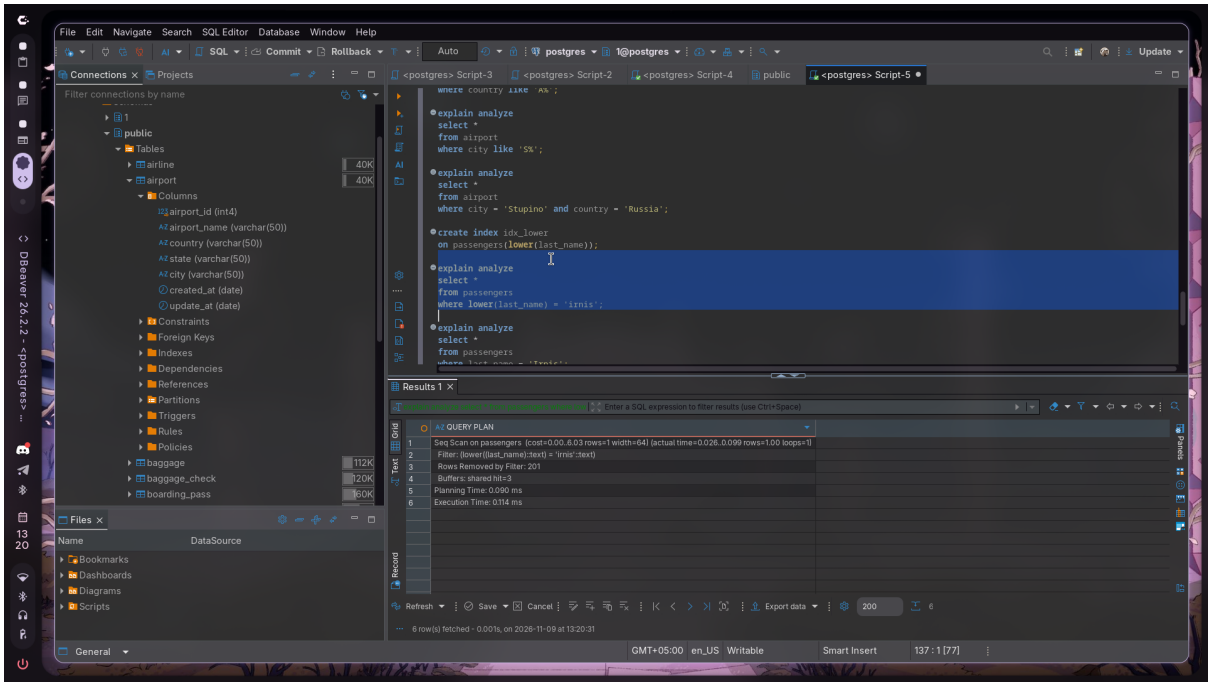




It used the seq scans.

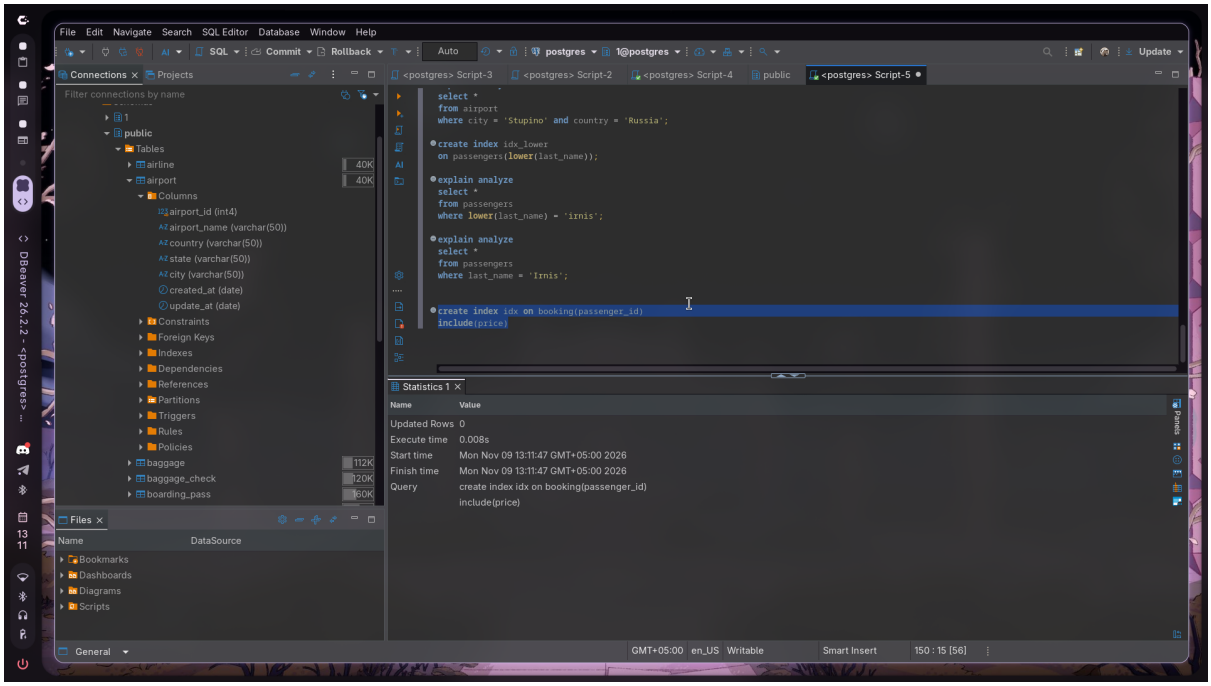
15)

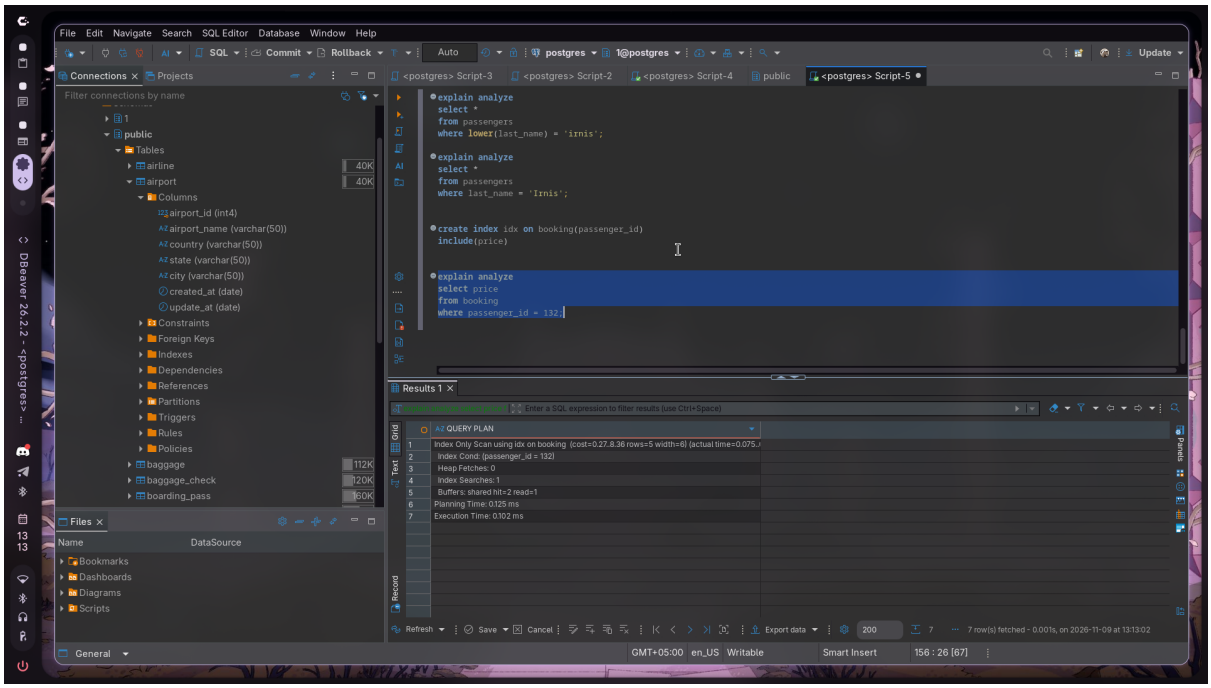




It used seq scan due to table didn't have that many rows and got only 20 rows so psql chose the seq scan.

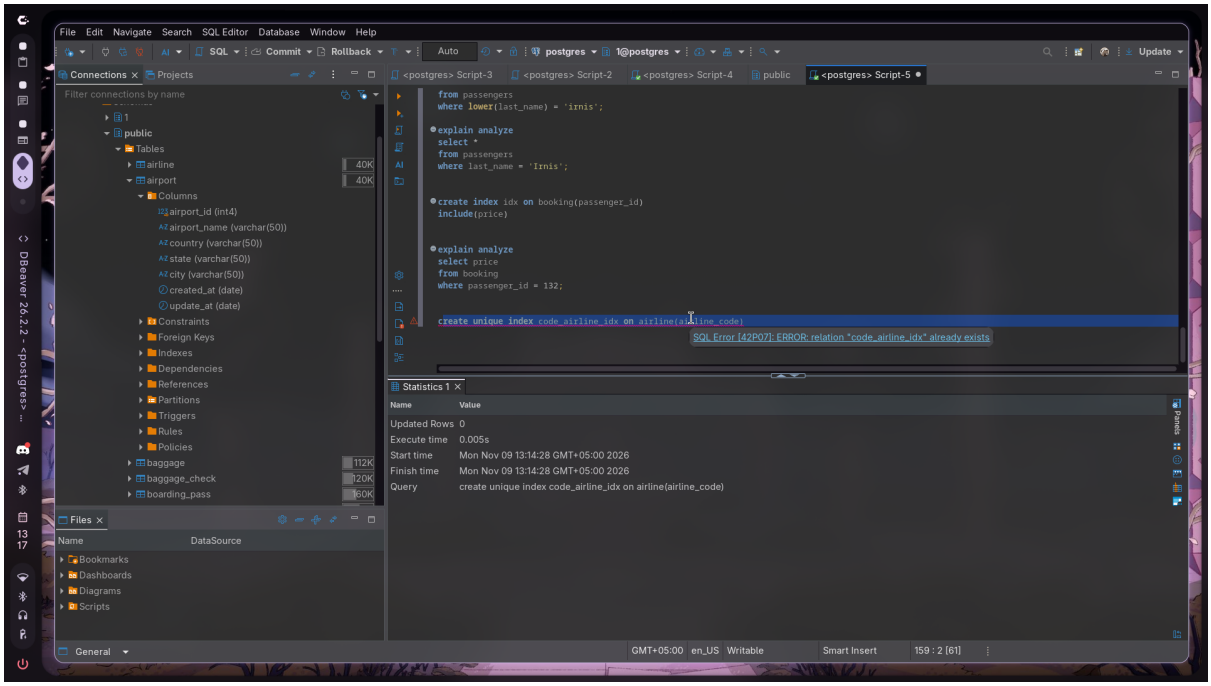
16)

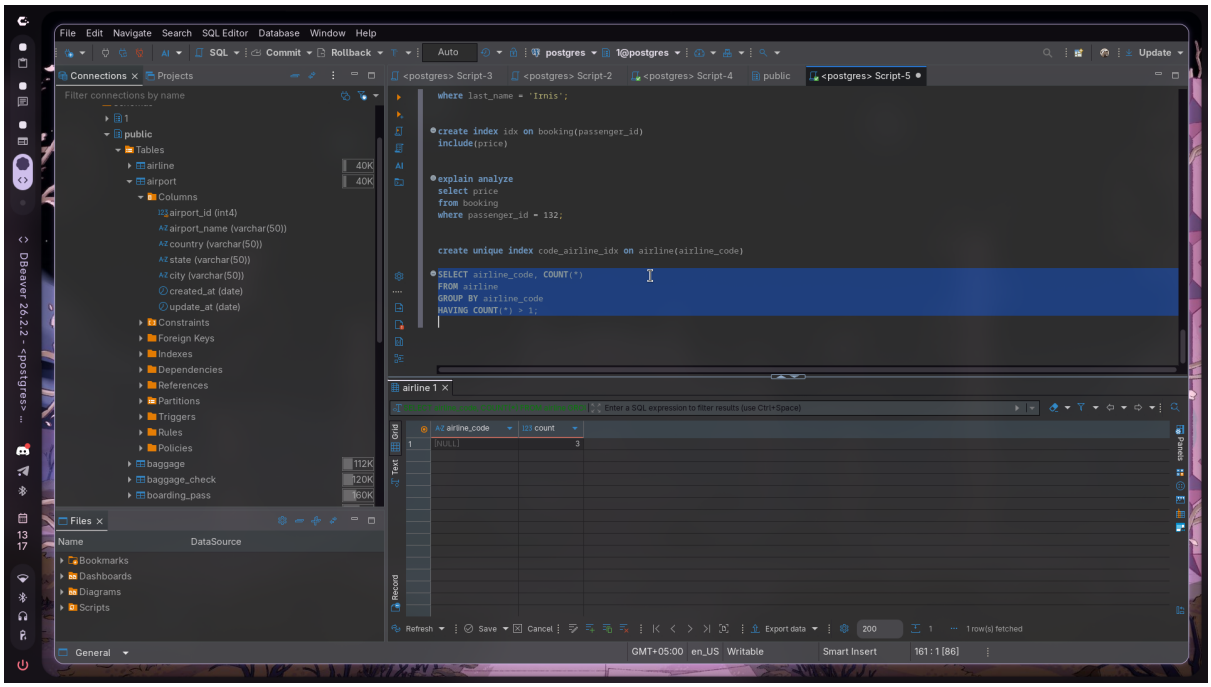




It is using index only scan because index contained all the information that it needed for the result in our case we needed price and passenger id so it accessed instantly because covering index was created with pass id and price.

17)


File Edit Navigate Search SQL Editor Database Window Help
Connections
Projects
<postgres> Script-3
<postgres> Script-2
<postgres> Script-4
public
<postgres> Script-5
from passengers
where lower(last_name) = 'irnis';
explain analyze
select *
from passengers
where last_name = 'Irnis';
create index idx on booking(passenger_id)
include(price)
explain analyze
select price
from booking
where passenger_id = 132;
create unique index code_airline_idx on airline(airline_code)
SQL Error [42P07]: ERROR: relation "code_airline_idx" already exists
Statistics 1
Name
Value
Updated Rows 0
Execute time 0.005s
Start time Mon Nov 09 13:14:28 GMT+05:00 2026
Finish time Mon Nov 09 13:14:28 GMT+05:00 2026
Query create unique index code_airline_idx on airline(airline_code)
Files
DataSource
Bookmarks
Dashboards
Diagrams
Scripts
General
GMT+05:00 en_US Writable Smart Insert 159 : 2 [61]


File Edit Navigate Search SQL Editor Database Window Help
Connections
Projects
<postgres> Script-3
<postgres> Script-2
<postgres> Script-4
public
<postgres> Script-5
where last_name = 'Irnis';
create index idx on booking(passenger_id)
include(price)
explain analyze
select price
from booking
where passenger_id = 132;
create unique index code_airline_idx on airline(airline_code)
SELECT airline_code, COUNT(*)
FROM airline
GROUP BY airline_code
HAVING COUNT(*) > 1;
airline 1
airline_code
count
[NULL] 3
Refresh Save Cancel Export data 200 1 row(s) fetched
Files
DataSource
Bookmarks
Dashboards
Diagrams
Scripts
General
GMT+05:00 en_US Writable Smart Insert 161 : 1 [86]